Assignment on bitwise operator

Bitise AND (&)

```
1: short int x=12, y=17 ,z;
    z=x & y
   printf("%d",z);


 2.short int x=12 , y=15 ,z
   z= x &y
  printf("%d",z);

3: x =-12 , y=15 ,z;
   z=x &y
  printf("%d",z);
```

//

Bitswise OR

```
1;short int x=15, y=24,z;
  z= x | y
  printf("%d",z);

2.int x=25, y=03,z=0;
  z= x | y
printf("%d",z);
```

//

Bitwise xor

```
 1:short int x=28, y=37 ,z;
  z=x  ^ y
printf("%d",z);

2: int x=5 , y= 3;
   int z;
   z= x ^ y;
  printf("% d", z);
```

//

1.On left shifting, the bits from the left are rotated and brought to the right and accommodated where there is empty space on the right?

1.True

2.False

//

Q: int main()
{
printf("%x\n", -1>>1);
return 0;
}

1: f f f f

2: 0 f f f

3: 0 0 0 0

4: f f f 0

//

Q:

#include<stdio.h>
int main()
{
printf("%x\n", -4<<2);
return 0;
}

A. fff0
B. 0
C. ffff
D. Error

//
If an *unsigned int* is 2 bytes wide then, What will be the output of the program ?

#include<stdio.h>

int main()
{
unsigned int m = 32;
printf("%x\n", ~m);
return 0;
}

1; f f f f

2. 0 0 0 0

3. f f d f

4. f f c f

//

q:If an *unsigned int* is 2 bytes wide then, What will be the output of the program ?

```
#include<stdio.h>

int main()
{
   unsigned int a=0xffff;
   ~a;
   printf("%x\n", a);
   return 0;
}
```

1; f f f f

2; 0 0 0 0

3; 0 0 f f

4: d d f d

//

Q:1. 24>>4 2.17<<2

//

Q:

```
  int num = 5;

  int result = ~num;

  printf(“%d”, result);
```

//

Q: int a = 5, b = 10;

```
  a ^= b;
  b ^= a;
  a ^= b;

 printf(“%d”,a,b);
```

//

Q:main(){

int a=-8 ,b =-9;

printf(“%d %d “, a<<1 , a>>1);

printf(“%d   %d “, b<<1 ,b>>1);

}

//

Q:

{

int a=20,b=0;

b= a<<3>>2;

printf(“%d   %d “,a ,b);

}

//

1;find out the result

printf(“%d “, 1<<5 >>5

printf("%d ", -15<<1);

printf(“%d”,5>>4);

printf(“%d “,1<<31 >>31);

//

Q:printf(“%d “,-15<< 1);

printf(“%d “,-15>>1);

printf(“%d”,-15>>2);

printf(“-64>>3);

printf(“%d”,-5>>4);

}